Autómata de la agenda

Caso práctico

Juan ha terminado la agenda de la TAREA-02A y funciona: añade contactos, los modifica, los borra y se vacía. **María** le pide, antes de darla por buena, el **diagrama de estado-transición ampliado**. —¿Para qué, si ya está hecha? —Para saber qué has programado de verdad. Dime cuántos estados tiene, qué botón se puede pulsar en cada uno y qué pasa en **todas** las casillas de la tabla, también en las que tu página no deja pulsar. Hay una línea en tu código que no se ejecuta nunca... hasta que alguien escribe la dirección a mano.

Este documento aplica el método de los diagramas de estado-transición ampliados a la solución de referencia de la **TAREA-02A (Agenda)**, tal como queda en `../PASO A PASO/paso5-solucion-completa/index.php`. El código **no** se escribió a partir de un autómata: aquí se hace el camino contrario, del código al autómata, que es la mejor manera de revisar una aplicación ya hecha. Todo lo que se afirma está comprobado con PHP 8.2.

1.- Cómo se lee el diagrama

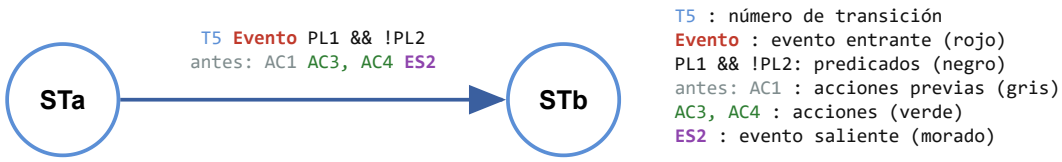


Figura 1. Leyenda del diagrama de estado-transición ampliado

2.- El problema, en resumen

- Una única página con un formulario (**nombre** y **teléfono**) que se envía a sí misma por **GET**.
- Si el nombre está vacío, se avisa y no se hace nada más.
- Si el nombre no está en la agenda, se **añade**; si ya está, se **modifica** su teléfono; y si el teléfono llega vacío, el contacto se **elimina**.
- La agenda se guarda en campos ocultos del formulario (`agenda[nombre]`), porque en este tema no hay sesiones ni base de datos.
- Mientras haya algún contacto se muestra la lista y un enlace **Vaciar**.

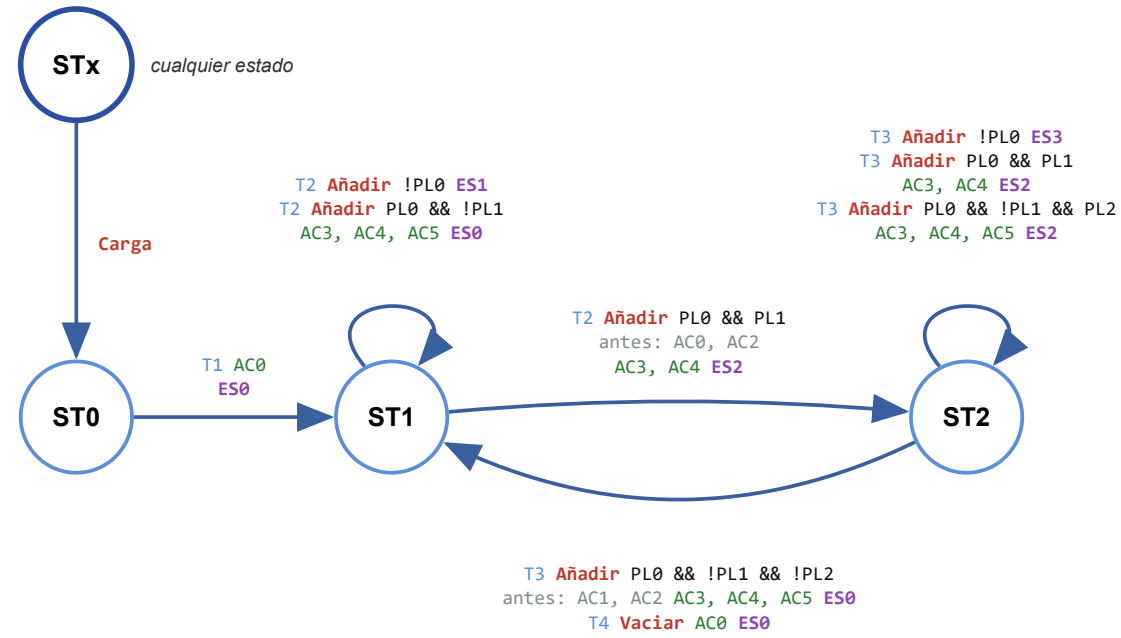
3.- Análisis

- **Eventos entrantes.** Tres: la **carga** de la página sin parámetros, el botón **Añadir Contacto** (`name='enviar'`) y el enlace **Vaciar** (`?limpiar=1`). El botón *Limpiar Campos* es un `type="reset"`: borra las cajas en el navegador y **no llega al servidor**, así que no es un evento del autómata.
- **Estados.** Dos situaciones en las que la página ofrece cosas distintas, más ST0: con la **agenda vacía** solo hay formulario; con la **agenda con contactos** hay además la lista y el enlace *Vaciar*.

Añadir, modificar y borrar no son estados: son lo que pasa **al cambiar** de uno a otro (o al quedarse).

- **Variables de estado.** Una sola: la agenda, un array asociativo `nombre => teléfono` que viaja en campos ocultos.
- **El estado es implícito.** No hay ningún campo `estado`: el servidor lo **deduce** de la variable de estado. Si la petición trae `agenda`, estaba en ST2; si no, en ST1 (o es la primera carga). Vale porque los dos estados solo se diferencian en eso: `count($almacenAgenda) > 0`.
- **Añadir y modificar son la misma transición.** El código no pregunta si el nombre existe: `$almacenAgenda[$nombre] = $telefono` hace las dos cosas. Por eso no hace falta un predicado «el contacto ya existe».
- **Un predicado que se evalúa después de las acciones.** PL2 («quedan contactos») decide el estado siguiente y solo se puede saber **después** de guardar o borrar: al borrar el único contacto se vuelve a ST1.
- **Una transición que hace el navegador.** El enlace *Vaciar* no envía la agenda (está fuera del formulario): la agenda se vacía porque **no viaja**. En el servidor, T4 no ejecuta ni una línea propia.

4.- Diagrama de estado-transición ampliado



antes: AC1 (recuperar la agenda de los campos ocultos) y AC2 (leer el nombre y el teléfono) en T3; AC0 y AC2 en T2.
PL2 (quedan contactos) se evalúa después de las acciones. Las páginas ES2 y ES3 llevan además AC6.
ST0 Carga · ST1 Agenda vacía · ST2 Agenda con contactos

Figura 2. Diagrama de la agenda

Desde cualquier estado, cargar la página sin parámetros (escribir la dirección, usar un marcador) lleva a ST0 y de ahí a ST1: **la agenda se pierde**, porque solo existe en la página que se tenía delante.

5.- Autómata

5.1.- Tabla de descriptores

Tipo	Abreviatura	Nombre	Significado
Evento entrante	EE0	Carga	petición sin `enviar` ni `limpiar`
Evento entrante	EE1	Añadir	botón *Añadir Contacto*: llegan `enviar`, `nombre`, `telefono` y, si hay, `agenda[...]`
Evento entrante	EE2	Vaciar	enlace *Vaciar*: llega solo `limpiar=1`
Evento saliente	ES0	agenda vacía	solo el formulario *Nuevo Contacto*
Evento saliente	ES1	agenda vacía con aviso	ES0 y «AVISO: el Nombre es obligatorio»
Evento saliente	ES2	agenda con contactos	la lista, el formulario (con los campos ocultos) y el enlace *Vaciar*
Evento saliente	ES3	agenda con contactos y aviso	ES2 y el aviso; la agenda no cambia
Estado	ST0	Carga	la petición llega sin nada: todavía no hay página
Estado	ST1	Agenda vacía	
Estado	ST2	Agenda con contactos	
Variable de estado	VE0	estado	**implícito**: no se guarda; es ST2 si `count($almacenAgenda) > 0` y ST1 si no
Variable de estado	VE1	`$almacenAgenda`	`['Ana Perez' => '986111222', ...]`, en los campos ocultos `agenda[Ana Perez]`
Predicado	PL0	hay nombre	`strlen($nombre) != 0` después de `trim()`
Predicado	PL1	hay teléfono	`strlen($telefono) != 0` después de `trim()`
Predicado	PL2	quedan contactos	`count($almacenAgenda) > 0`, **después** de las acciones
Acción	AC0	agenda vacía	`$almacenAgenda = array()` (la petición no trae `agenda`)
Acción	AC1	recuperar la agenda	`$almacenAgenda = $_GET['agenda']`
Acción	AC2	leer el formulario	`trim()` del nombre y del teléfono
Acción	AC3	normalizar el nombre	`ucwords()`: `ana perez` es `Ana Perez`
Acción	AC4	guardar	`$almacenAgenda[$nombre] = $telefono` (añade o modifica)
Acción	AC5	eliminar	`unset($almacenAgenda[$nombre])`
Acción	AC6	guardar el estado	un campo oculto `agenda[nombre]` por contacto, dentro del formulario

5.2.- Tabla de eventos-estados ampliada

	Carga	Añadir	Vaciar
ST0	T1	---	---
ST1	T1	T2	---
ST2	T1	T3	T4

Las celdas `---` no pueden darse con la página: en ST0 no hay botones y en ST1 no se dibuja el enlace *Vaciar*. Si alguien escribe `?limpiar=1` con la agenda vacía, el resultado es el de T1 (comprobado). Como el estado es implícito, el servidor no distingue ST0 de ST1: en los dos la petición llega sin agenda.

5.3.- Tabla de transiciones

#	AEs antes	PLs	Nuevo ST	AEs después	ESs
T1	---	---	ST1	AC0	ES0
T2	AC0, AC2	`!PL0`	ST1	---	ES1
T2	AC0, AC2	`PL0 && PL1`	ST2	AC3, AC4	ES2 [AC6]
T2	AC0, AC2	`PL0 && !PL1`	ST1	AC3, AC4, AC5	ES0
T3	AC1, AC2	`!PL0`	ST2	---	ES3 [AC6]
T3	AC1, AC2	`PL0 && PL1`	ST2	AC3, AC4	ES2 [AC6]
T3	AC1, AC2	`PL0 && !PL1 && PL2`	ST2	AC3, AC4, AC5	ES2 [AC6]
T3	AC1, AC2	`PL0 && !PL1 && !PL2`	ST1	AC3, AC4, AC5	ES0
T4	AC0	---	ST1	---	ES0

- **Borrar es guardar y quitar.** Cuando el teléfono llega vacío, el código primero guarda el contacto sin teléfono (AC4) y en la línea siguiente lo quita (AC5). Si el contacto no existía, el resultado es que no pasa nada.
- **T2 con `PL0 && !PL1`**: intentar borrar con la agenda vacía. No hay aviso ni cambio: la página vuelve a salir igual.
- **T4 no tiene acciones propias.** La agenda queda vacía por AC0: la petición no la trae.

5.4.- Comprobación de predicados

T3 (con **x** = no se evalúa o no puede cambiar el resultado):

PL0	PL1	PL2	aviso	guarda	borra y quedan	borra y se vacía
0	x	x	1	0	0	0
1	1	x	0	1	0	0
1	0	1	0	0	1	0
1	0	0	0	0	0	1

En cada fila hay exactamente un 1: los caminos son **completos** y **excluyentes**. En la segunda fila PL2 es siempre cierto (se acaba de guardar un contacto), por eso no hace falta mirarlo.

T2:

PL0	PL1	aviso	guarda (ST2)	no cambia (ST1)
0	x	1	0	0
1	1	0	1	0
1	0	0	0	1

6.- Del autómata al código

El autómata está en `index.php`, pero **repartido**: no hay `switch` ni tabla. Cada pieza está aquí (líneas de `../PASO A PASO/paso5-solucion-completa/index.php`):

Líneas	Qué es en el autómata
18-21	AC1 o AC0: recuperar la variable de estado (y, con ella, el estado)
39	¿evento *Añadir*?
41-42	AC2
43-44	`!PL0` y el aviso de ES1 / ES3
46-47	AC3 y AC4
48	`!PL1` y AC5
53-58	¿evento *Vaciar* con contactos? (ver el apartado 7)
65 y 110	PL2: se dibujan la lista y el enlace *Vaciar* (ES2 / ES3) o no (ES0 / ES1)
99-102	AC6

La misma aplicación, escrita **a partir** del autómata, quedaría así (el estado se sigue deduciendo de la agenda):

```
const ST1 = 1, ST2 = 2;                                    // agenda vacía, agenda con
contactos
const EE_CARGA = 0, EE_ANADIR = 1, EE_VACIAR = 2;
$automata = [                    // Carga Añadir Vaciar
    ST1 => [1, 2, null],
    ST2 => [1, 3, 4],
];

$agenda = is_array($_GET['agenda'] ?? null) ? $_GET['agenda'] : [];   // AC1 o AC0
$estado = count($agenda) > 0 ? ST2 : ST1;                              // VE0, implícita
$evento = match (true) {
    isset($_GET['enviar']) => EE_ANADIR,
    isset($_GET['limpiar']) => EE_VACIAR,
    default => EE_CARGA,
};
$aviso = '';
switch ($automata[$estado][$evento] ?? 1) {              // null ("---") se trata como T1
    case 2:                                              // ---- T2 y T3: Añadir
    case 3:
        $nombre = is_string($_GET['nombre'] ?? null) ? trim($_GET['nombre']) : '';        //
AC2
        $telefono = is_string($_GET['telefono'] ?? null) ? trim($_GET['telefono']) : '';
        if ($nombre === '') {                                          // !PL0
            $aviso = 'AVISO: el Nombre es obligatorio';
        } elseif ($telefono !== '') {                                  // PL0 && PL1
            $agenda[ucwords($nombre)] = $telefono;                     // AC3, AC4
        } else {                                                       // PL0 && !PL1
            unset($agenda[ucwords($nombre)]);                          // AC3, AC5
        }
        break;
    case 4:                                              // ---- T4: Vaciar
        $agenda = [];                                                  // ahora sí es una
acción del servidor
        break;
}
$estado = count($agenda) > 0 ? ST2 : ST1;                // PL2: el estado siguiente
```

Tres diferencias con la solución de referencia, todas a favor de la versión con autómata:

- **Un evento por petición.** El `match` elige uno solo; en el original son dos `if` seguidos y pueden cumplirse los dos.
- **La variable de estado se comprueba** al recuperarla (`is_array`), y los datos del formulario también (`?? null` e `is_string`).
- **T4 es una acción del servidor** (`$agenda = []`), no una consecuencia de lo que el enlace deja de enviar.

7.- Lo que el autómata descubre

La tabla de eventos-estados obliga a pensar en **todas** las combinaciones, también en las que la página no deja hacer. El navegador solo ofrece los botones de la página; quien escribe la dirección a mano, no. Estas peticiones están probadas contra `paso5-solucion-completa` con PHP 8.2:

Petición	Qué es en el autómata	Qué pasa
`?limpiar=1&agenda[Ana Perez]=986111222`	*Vaciar* en ST2 **con** la agenda (lo que mandaría un botón *Vaciar* puesto dentro del formulario)	`unset($almacenAgenda)` (línea 57) destruye la variable de estado entera y `count()` da un **error fatal** en la línea 65
`?nombre=xan&telefono=1&enviar=x&limpiar=1`	dos eventos en la misma petición	se añade el contacto, la agenda ya no está vacía, se ejecuta la línea 57 y el mismo **error fatal**
`?enviar=x`	*Añadir* sin sus datos	dos *Warning* (`Undefined array key`) y dos *Deprecated* (`trim(null)`), y después el aviso del nombre
`?nombre[]=a&telefono=1&enviar=x`	un dato que no es un texto	**error fatal** en `trim()` (línea 41)
`?agenda=hola`	una variable de estado que no es un array	**error fatal** en `count()` (línea 65)
contacto `ana "jefa"` y después otro	AC6 escribe el estado sin `htmlspecialchars()`	las comillas dobles cortan el atributo `name` del campo oculto: en la petición siguiente ese contacto **ha desaparecido**
contacto `O'Brien`	la lista (ES2) tampoco escapa	el estado se salva, pero la lista enseña solo `O`; y un teléfono `6'1` se queda en `6`

Las dos primeras filas explican el `FIXME (pendiente)` de la línea 57: con el enlace de la página esa línea no se ejecuta nunca (el enlace no manda la agenda, así que `count($almacenAgenda) > 0` es falso), pero es código que **parece** hacer T4 y en realidad rompe la página. Lo correcto es `$almacenAgenda = array();`.

Otro detalle que sale al normalizar: `ucwords()` solo pone en mayúscula la primera letra de cada palabra y deja el resto como está. `ANA PEREZ` y `Ana Perez` son dos contactos distintos (comprobado).

8.- Pruebas: un camino por fila

Los seis casos de prueba del `README.md` de la tarea recorren seis de las nueve filas de la tabla de transiciones. El autómata dice cuáles faltan (los tres últimos):

Camino	Petición	Resultado
T1	abrir `index.php`	solo el formulario
T2 `!PL0`	nombre vacío (o solo espacios), teléfono `123`	aviso; agenda vacía (caso 1)
T2 `PL0 && PL1`	`ana perez` / `986111222`	*Ana Perez — 986111222* y aparece *Vaciar* (caso 2)
T3 `PL0 && PL1`	`brais` / `600`; después `ana perez` / `111`	se añade Brais; se modifica Ana Perez (casos 3 y 4: el mismo camino con datos distintos)
T3 `PL0 && !PL1 && PL2`	`brais` y teléfono vacío	desaparece Brais, queda Ana Perez (caso 5)
T4	pulsar *Vaciar*	agenda vacía y sin *Vaciar* (caso 6)
T3 `!PL0`	con contactos, nombre vacío	aviso y **la agenda sigue igual**
T2 `PL0 && !PL1`	con la agenda vacía, `ana` y teléfono vacío	no pasa nada: ni aviso ni contacto
T3 `PL0 && !PL1 && !PL2`	con un solo contacto, su nombre y teléfono vacío	agenda vacía y **desaparece el enlace Vaciar**: vuelve a ST1

Y un caso más dentro de T3 `PL0 && !PL1 && PL2`: borrar un nombre que **no está** (`carme` y teléfono vacío). No cambia nada y no se avisa.

9.- Para practicar

- Añade un predicado PL3 «el teléfono solo tiene cifras» y un aviso si no se cumple. ¿Qué filas de la tabla de transiciones cambian? ¿Hace falta un evento saliente nuevo?
- Convierte el enlace *Vaciar* en un botón `submit` **dentro** del formulario. ¿Qué llega ahora al servidor? Reescribe T4 con sus acciones.
- Añade un estado *Confirmar* entre ST2 y ST1: al pulsar *Vaciar* se pregunta «¿Seguro?» con dos botones. Dibuja el diagrama y completa la tabla de eventos-estados. ¿Sigue pudiendo ser implícito el estado?
- Haz que el estado viaje en un campo oculto `estado`. ¿Qué tendría que comprobar el servidor al recibirlo?

Autoevaluación

- ¿Por qué añadir y modificar no son dos transiciones distintas? **Porque el código hace lo mismo en los dos casos (`$almacenAgenda[$nombre] = $telefono`) y la página que sale es la misma. Un array asociativo no distingue entre crear una clave y cambiar su valor.**
- ¿Dónde está guardado el estado de la agenda? **En ningún sitio: se deduce de la variable de estado. Si llegan campos `agenda[...]` hay contactos (ST2); si no, la agenda está vacía (ST1).**
- ¿Por qué PL2 se evalúa después de las acciones? **Porque pregunta si quedan contactos, y eso solo se sabe después de guardar o de borrar. Al borrar el único contacto, la aplicación vuelve a ST1 y el enlace «Vaciar» desaparece.**
- ¿Qué hace el servidor en T4? **Nada propio: el enlace «Vaciar» no envía la agenda y el servidor crea una vacía (AC0), igual que en la primera carga. La línea `unset($almacenAgenda)` no llega a ejecutarse con la página... y si se ejecuta, da un error fatal.**
